

Calcolatori Elettronici T
Ing. Informatica

Traccia soluzione
15 Febbraio 2017

Esercizio 1

Dispositivi e segnali presenti nel sistema:

RAM_128	mappata da F8000000h:FFFFFFFFh, 4 banchi da 32 MB
EPROM_L	mappata da 00000000h:1FFFFFFFFh, 4 banchi da 128 MB
EPROM_H	mappata da 20000000h:2FFFFFFFFh, 4 banchi da 64 MB

Le quattro periferiche in **INPUT_k**, **k=[A,B,C,D]** sono mappate, rispettivamente, agli indirizzi:

CS_INPUT_PORT_A	mappato a 80000000h
CS_INPUT_PORT_B	mappato a 80000001h
CS_INPUT_PORT_C	mappato a 80000002h
CS_INPUT_PORT_D	mappato a 80000003h

Infine, è presente un segnale di reset software utilizzato all'interno dall'interrupt handler:

CS_RESET_SW	mappato a 80000004h
-------------	---------------------

Segnali di decodifica di memorie, periferiche e segnali:

$CS_RAM_512_H_0 = BA31 \cdot BA30 \cdot BE0 \cdot (IN_HANDLER^* + IN_HANDLER \cdot INT_INPUT_PORT_A_READY)$
 $CS_RAM_512_H_1 = BA31 \cdot BA30 \cdot BE1 \cdot (IN_HANDLER^* + IN_HANDLER \cdot INT_INPUT_PORT_B_READY)$
 $CS_RAM_512_H_2 = BA31 \cdot BA30 \cdot BE2 \cdot (IN_HANDLER^* + IN_HANDLER \cdot INT_INPUT_PORT_C_READY)$
 $CS_RAM_512_H_3 = BA31 \cdot BA30 \cdot BE3 \cdot (IN_HANDLER^* + IN_HANDLER \cdot INT_INPUT_PORT_D_READY)$

$CS_RESET_SW = BA31 \cdot BA30^* \cdot BA2 \cdot MEMRD$ (meglio se condizionato da $IN_HANDLER$)

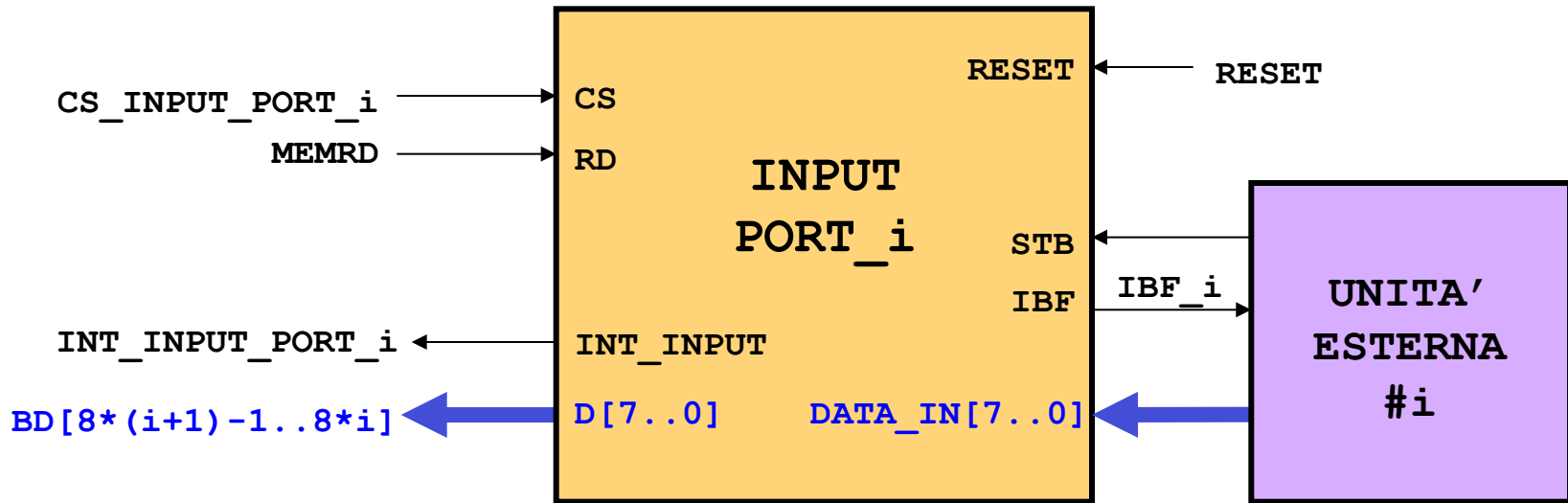
$CS_INPUT_PORT = BA31 \cdot BA30^* \cdot BA2^*$
 $CS_INPUT_PORT_A = CS_INPUT_PORT \cdot INT_INPUT_PORT_A_READY \cdot BE0 \quad ; \quad 80000000h$
 $CS_INPUT_PORT_B = CS_INPUT_PORT \cdot INT_INPUT_PORT_B_READY \cdot BE1 \quad ; \quad 80000001h$
 $CS_INPUT_PORT_C = CS_INPUT_PORT \cdot INT_INPUT_PORT_C_READY \cdot BE2 \quad ; \quad 80000002h$
 $CS_INPUT_PORT_D = CS_INPUT_PORT \cdot INT_INPUT_PORT_D_READY \cdot BE3 \quad ; \quad 80000003h$

$CS_EPROM_L_0 = BA31^* \cdot BA29^* \cdot BE0$
 $CS_EPROM_L_1 = BA31^* \cdot BA29^* \cdot BE1$
 $CS_EPROM_L_2 = BA31^* \cdot BA29^* \cdot BE2$
 $CS_EPROM_L_3 = BA31^* \cdot BA29^* \cdot BE3$

$CS_EPROM_H_0 = BA31^* \cdot BA29 \cdot BE0$
 $CS_EPROM_H_1 = BA31^* \cdot BA29 \cdot BE1$
 $CS_EPROM_H_2 = BA31^* \cdot BA29 \cdot BE2$
 $CS_EPROM_H_3 = BA31^* \cdot BA29 \cdot BE3$

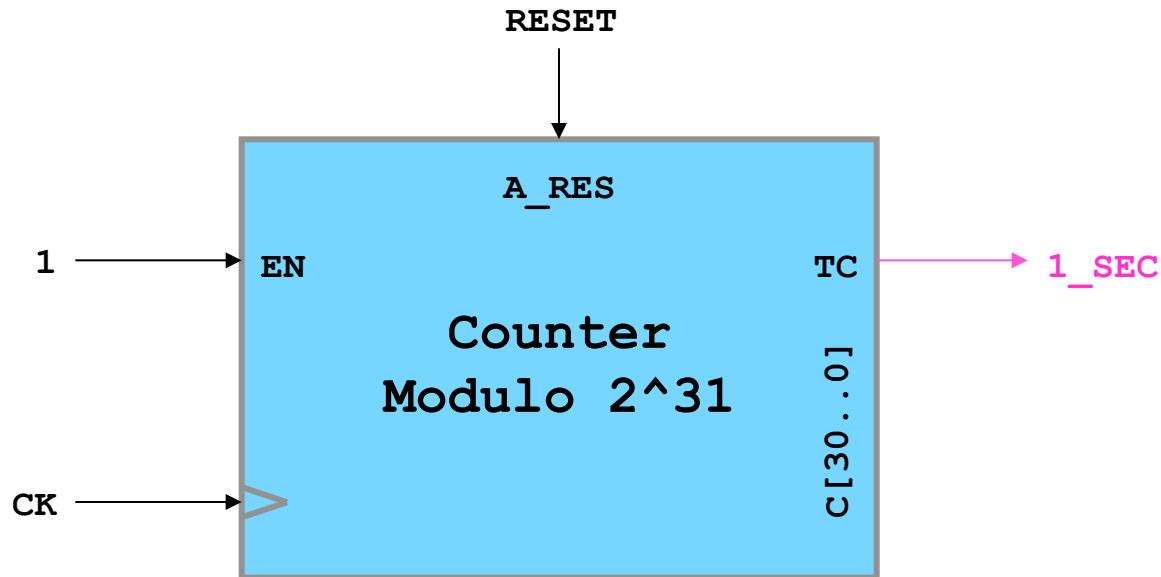
Il segnale $IN_HANDLER$, se asserito, indica che è stata eseguita una lettura dalle porte in input. I segnali $INT_INPUT_PORT_i_READY$ con $i=[A,B,C,D]$ codificano, allo scattare di ogni secondo, quali porte in input erano pronte in quel momento per un trasferimento. Tutti i segnali saranno sintetizzati nelle pagine successive mediante l'utilizzo di opportune reti logiche.

Le 4 periferiche **INPUT_PORT_i**, $i=[A,B,C,D]$ sono connesse a 4 bus, rispettivamente, **BD[7..0]** (porta A), **BD[15..8]** (porta B), **BD[23..16]** (porta C) e **BD[31..24]** (porta D). Nella figura gli indici $i=[A,B,C,D]$ corrispondono, rispettivamente, a $i=[0,1,2,3]$.



In accordo alle specifiche del testo le letture dalle porte in input disponibili a eseguire un trasferimento dovranno avvenire, quando ciò è possibile, ogni secondo. Il riferimento temporale, segnale **1_SEC**, è fornito dal segnale di *terminal count* **TC** di un contatore modulo 2^{31} . Al fine di minimizzare il numero di istruzioni eseguite dall'interrupt handler e il tempo di risposta, si procederà *condizionando* i CS delle porte in input e delle RAM, nella quali dovranno essere scritti i dati letti dalle periferiche abilitate, con i segnali di interrupt generati dalle medesime porte dopo un appropriato campionamento (tali segnali, campionati, sono denominati **INT_INPUT_PORT_i_READY** con $i=[A,B,C,D]$).

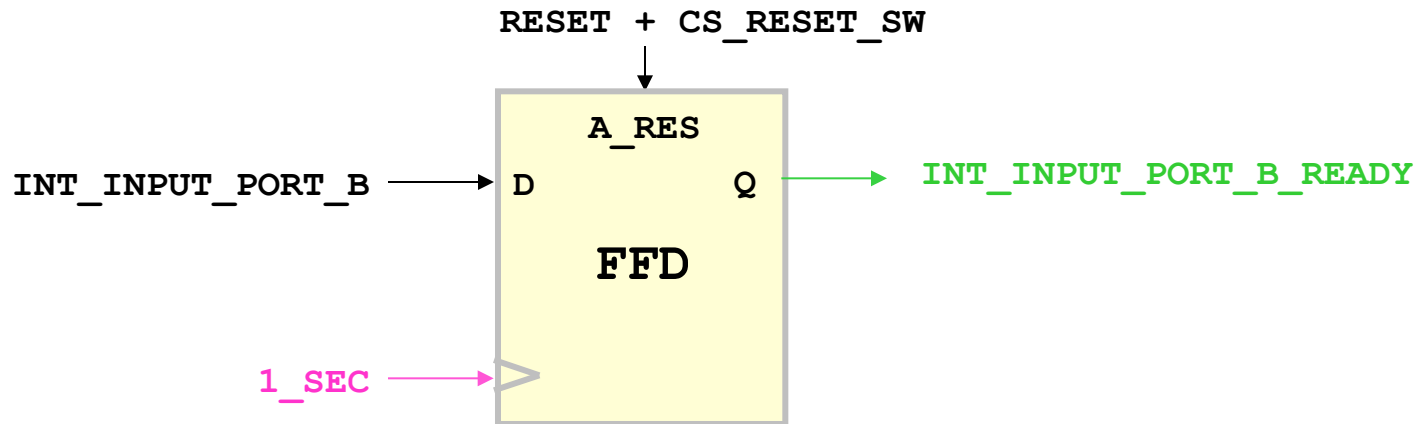
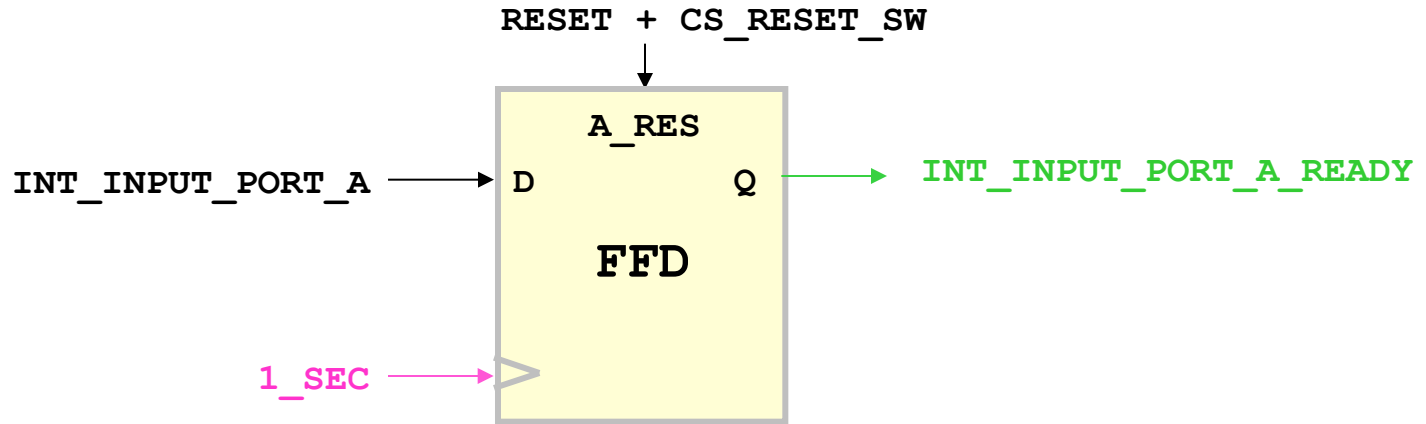
Essendo il clock del processore a 2 GHz, mediante un contatore modulo 2^{31} sarà generato ogni secondo un segnale denominato **1_SEC** utilizzato per campionare i segnali di interrupt generati dalle porte in input e per invertire lo stato del **LED**. Come richiesto dal testo del problema, il contatore modulo 2^{31} è inizializzato all'avvio agendo sul segnale asincrono di reset.

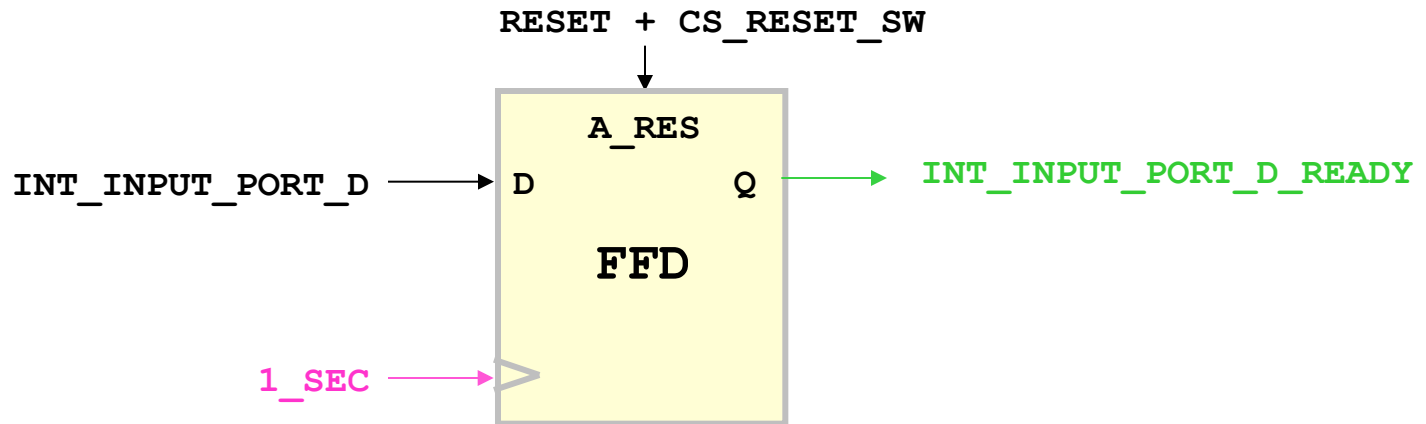
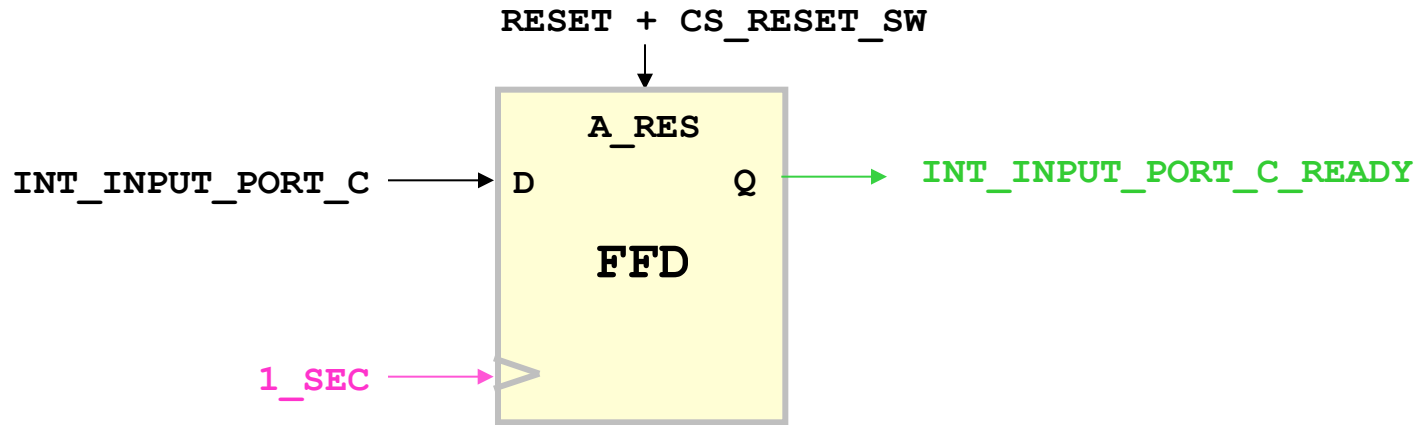


Il fronte di salita del segnale **1_SEC** è utilizzato per:

- campionare, ogni secondo, i segnali di interrupt generati dalle porte in input
- invertire, ogni secondo, lo stato di un **LED** nei casi in cui nessuna porta in input sia pronta per trasferire un dato

I segnali di interrupt generati dalle quattro periferiche in input sono campionati ogni secondo mediante i quattro FFD seguenti. Tali FFD sono resettati agendo sui comandi sincroni all'avvio mediante il segnale di **RESET** e mediante il comando software **CS_RESET_SW** mappato all'indirizzo **80000004h** attivato dall'interrupt handler.



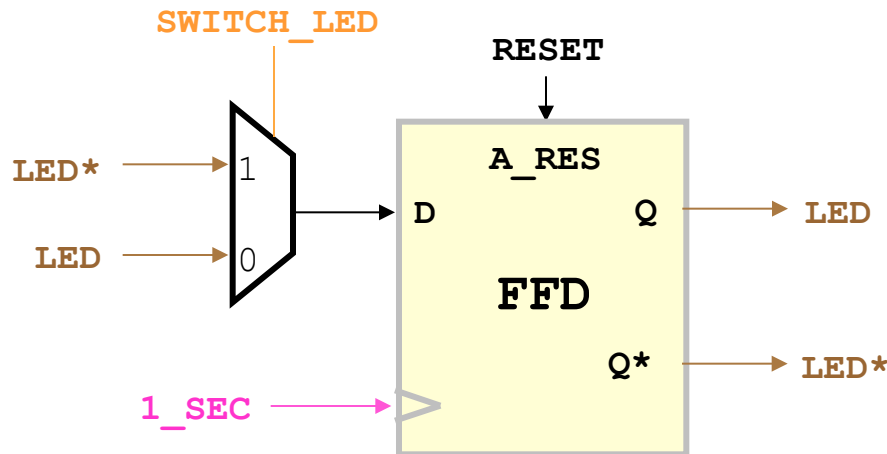


Il segnale di interruzione inviato al DLX è l'OR tra i quattro segnali di interrupt generati dalle porte in input e opportunamente campionati, ogni secondo, dai quattro FFD precedenti. Pertanto,

$$\text{INT_TO_DLX} = \text{INT_INPUT_PORT_A_READY} + \text{INT_INPUT_PORT_B_READY} + \text{INT_INPUT_PORT_C_READY} + \text{INT_INPUT_PORT_D_READY}$$

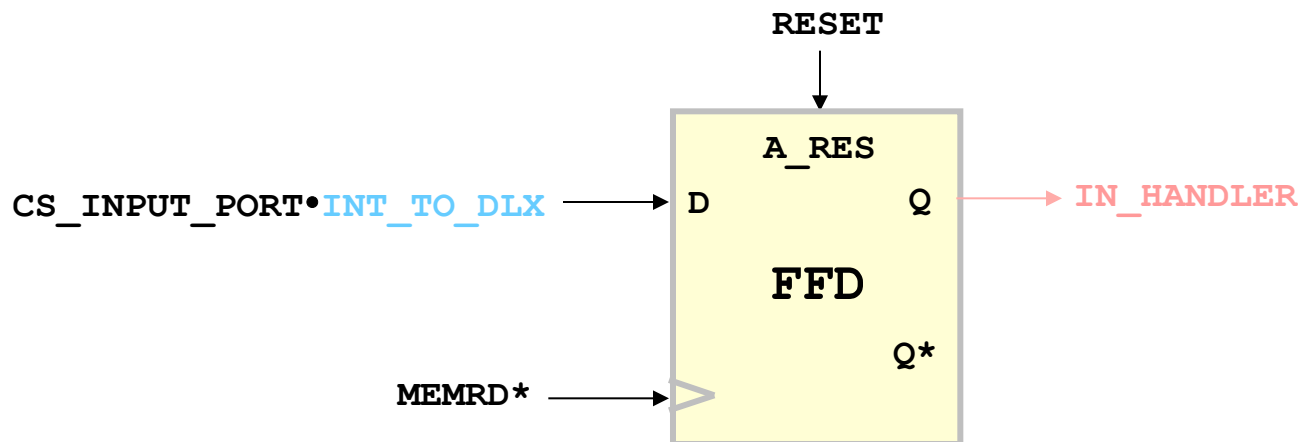
L'unico segnale di interrupt inviato al DLX, mostrato nella pagina precedente, si asserisce (ogni secondo) soltanto se vi è almeno una porta in input pronta. In caso contrario, come richiesto dal testo, il segnale INT inviato al DLX non si asserirà e pertanto, in tali circostanze, il normale flusso di esecuzione delle istruzioni non sarà interrotto per mettere in esecuzione l'interrupt handler. Il segnale **LED**, che pilota un dispositivo luminoso è ottenuto dalla rete seguente. All'avvio, come richiesto, il LED è spento. Il segnale **LED inverte il proprio stato**, sul fronte di salita del segnale **1_SEC**, solo **se nessuna** delle porte in input è in grado di trasferire un dato. Tale evento è codificato dal segnale **SWITCH_LED**:

$$\text{SWITCH_LED} = \text{NOT}(\text{INT_INPUT_PORT_A} + \text{INT_INPUT_PORT_B} + \text{INT_INPUT_PORT_C} + \text{INT_INPUT_PORT_D})$$

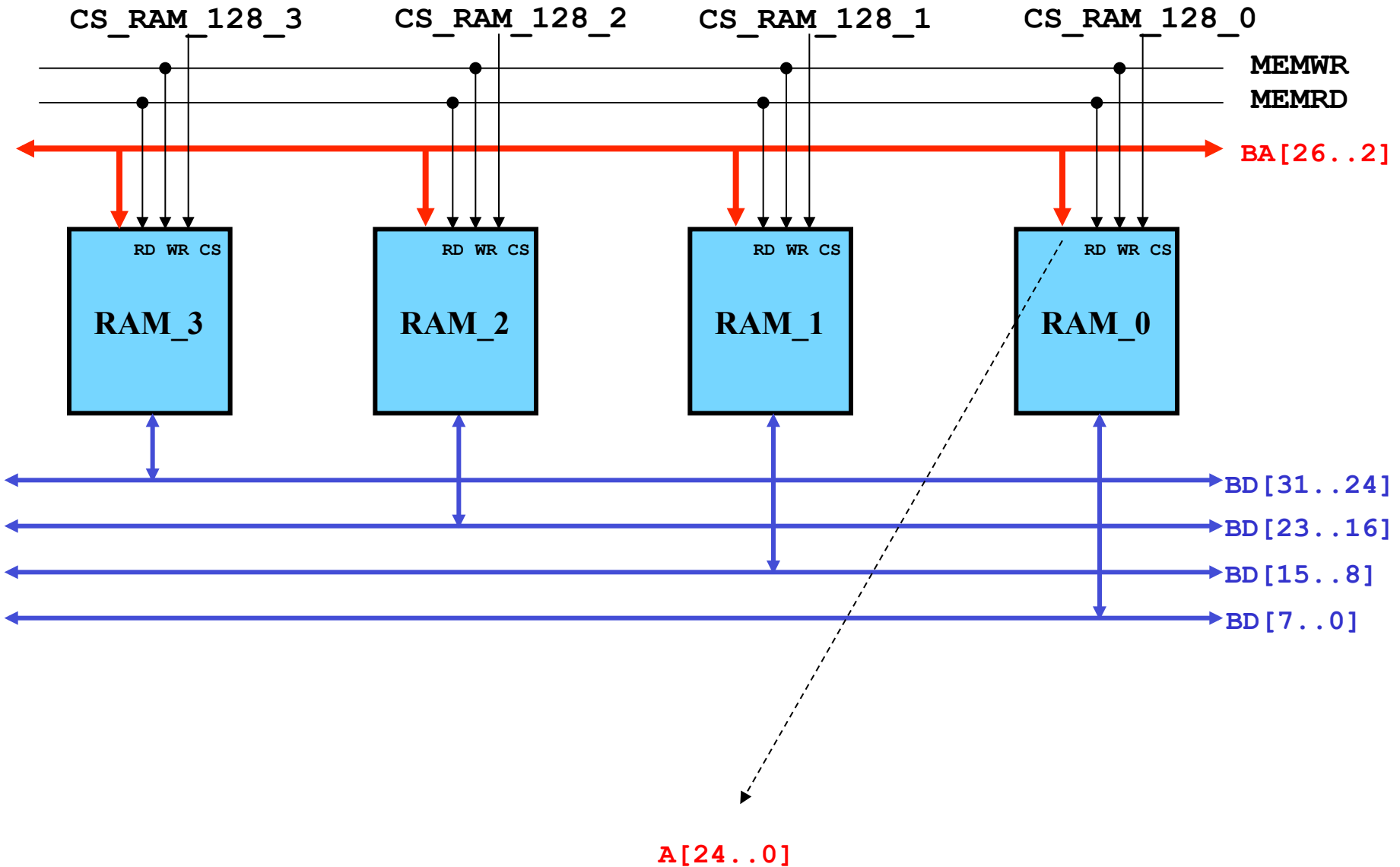


Come richiesto dal testo, al fine di ridurre il numero di istruzioni eseguite dall'interrupt handler e il tempo di risposta, si adotta la strategia descritta nella pagina successiva.

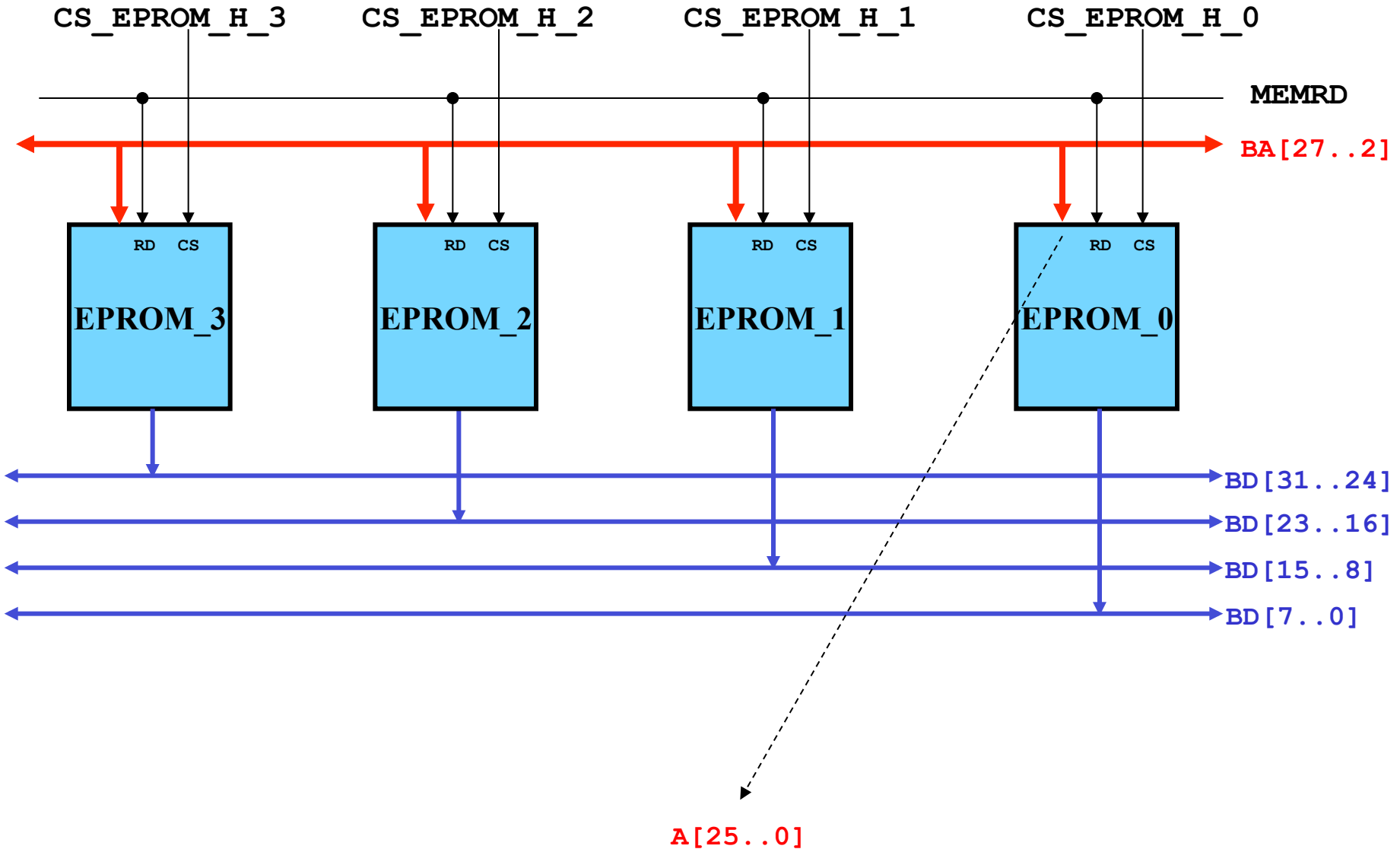
Essendo gli accessi alle periferiche possibili (vedi testo) solo da parte di istruzioni appartenenti all'interrupt handler, si provvede a generare un segnale, denominato **IN_HANDLER**, che quando è asserito codifica che l'handler ha effettuato un accesso alle porte in input. Il segnale **IN_HANDLER** condiziona la scrittura nelle RAM in modo che siano aggiornate solo le memorie associate alle porte in input dalle quali è stato effettivamente possibile leggere (ovvero, le porte per le quali uno specifico segnale **INT_INPUT_PORT_i_READY** è asserito). Il segnale **CS_RESET_SW**, attivato dall'ultima istruzione dell'interrupt handler, imposterà a 0 i quattro segnali **INT_INPUT_PORT_i_READY**, con $i=[A,B,C,D]$. Al termine della medesima istruzione sarà posto a 0 il segnale **IN_HANDLER**. Il segnale **CS_INPUT_PORT** in ingresso al flip-flop D è condizionato con **INT_TO_DLX** (OR tra i 4 **INT_INPUT_PORT_i_READY**) al fine di non impostare a uno il segnale **IN_HANDLER** in assenza di almeno un segnale interrupt (campionato). Il condizionamento con **INT_TO_DLX** è opzionale ma potrebbe essere opportuno. Perché? Per un motivo analogo, sarebbe opportuno condizionare **CS_RESET_SW** con **IN_HANDLER**. Perché?



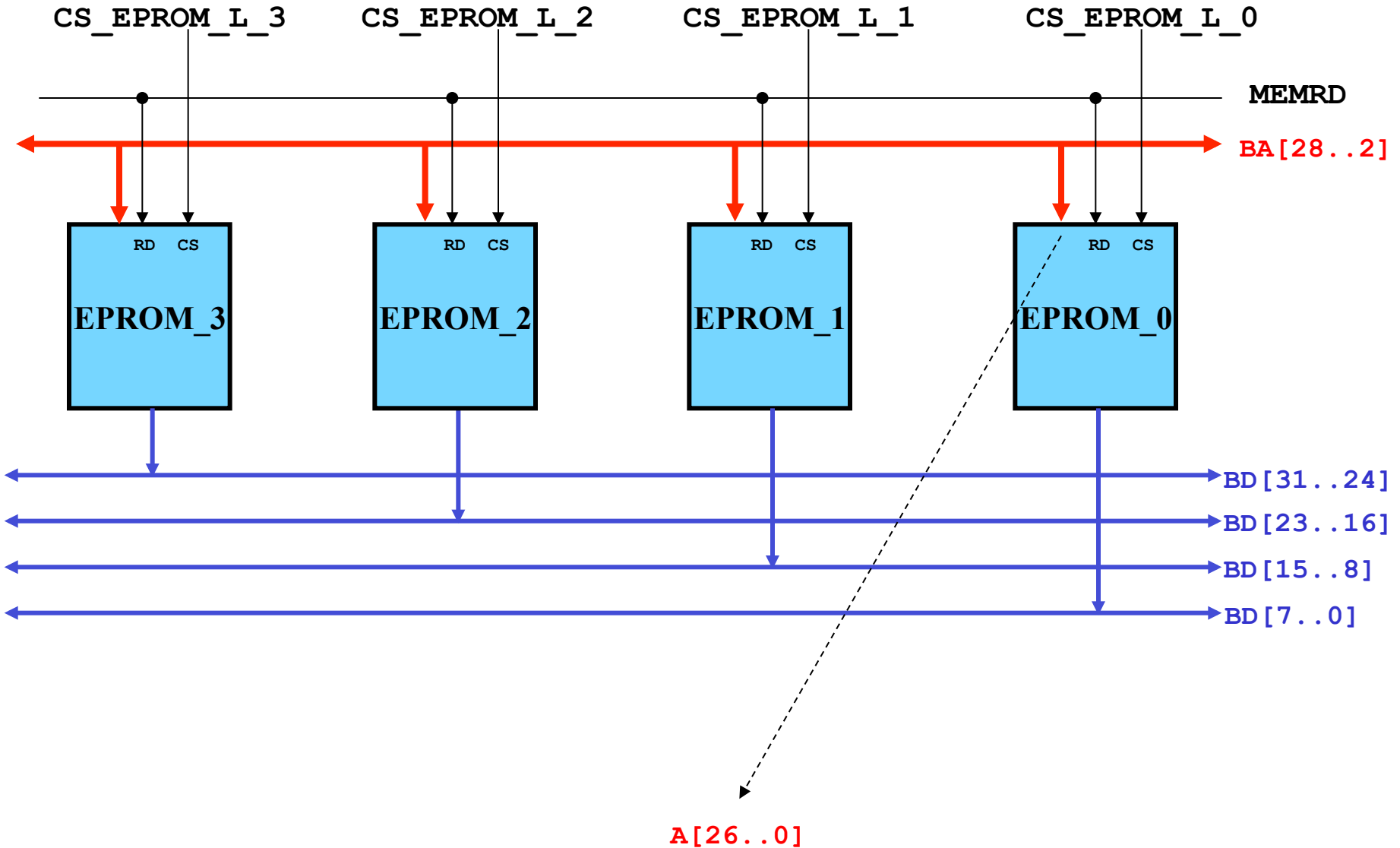
Interfacciamento RAM_128



Interfacciamento EPROM_H



Interfacciamento EPROM_L



Esercizio 2

Rif. lucidi/lezioni.

Esercizio 3

Rif. lucidi/lezioni.